Python Engineering - NumPy Quick Note

Adding rows to NumPy arrays: append, vstack, concatenate, and preallocation

Support note for **week_02_data_structures -> class_07_numpy_arrays**. Goal: understand how to grow a trajectory inside a loop and why NumPy does not behave like a Python list.

1. Core difference: list.append() vs np.append()

Python lists grow dynamically. An **ndarray** uses a fixed-size memory block. Therefore `np.append()` does not modify the original array; it creates a new one.

```
import numpy as np

A = np.array([1, 2, 3])
np.append(A, 4)

print(A)
# [1 2 3]

A = np.append(A, 4)
# now A is [1 2 3 4]
```

Key idea: with NumPy you usually reassign the result: `A = np.append(A, ...)`.

2. Warning: np.append() without axis flattens

```
A = np.array([[0.0, 50.0, 0.0]])
new_state = np.array([5.0, 49.019, 0.1])

B = np.append(A, new_state)
# B becomes 1D
```

If `axis` is omitted, NumPy flattens inputs before concatenating them.

3. np.append(..., axis=0)

```
trajectory = np.array([[h0, v0, t0]])
new_state = np.array([[h_new, v_new, t_new]])

trajectory = np.append(
    trajectory,
    new_state,
    axis=0
)
```

4. Turning a 1D row into 2D

```
new_state = np.array([h_new, v_new, t_new])
new_state_2d = new_state.reshape(1, 3)

# equivalent:
new_state_2d = new_state[np.newaxis, :]
```

5. np.vstack(): intuitive row stacking

```
trajectory = np.array([[h0, v0, t0]])
new_state = np.array([h_new, v_new, t_new])

trajectory = np.vstack((trajectory, new_state))
```

`vstack` means vertical stack. It is very readable when adding one state below another.

6. np.concatenate(): general solution

```
new_state = np.array([[h_new, v_new, t_new]])
```

```
trajectory = np.concatenate(
    (trajectory, new_state),
    axis=0
)
```

7. What does axis mean?

Operation	Interpretation
axis=0	grow by rows; add states underneath
axis=1	grow by columns; add variables beside existing ones

8. Inside our Euler while loop

```
trajectory = np.array([[h0, v0, t0]])

while trajectory[-1, 0] >= 0:
    h, v, t = trajectory[-1, :]
    h_new, v_new, t_new = euler_step(h, v, t, g, dt)
    new_state = np.array([h_new, v_new, t_new])
    trajectory = np.vstack((trajectory, new_state))
```

This works and is excellent for learning, but every vstack call allocates a new array and copies previous data.

9. Recommended when N is UNKNOWN

```
trajectory = [(h0, v0, t0)]

while trajectory[-1][0] >= 0:
    h, v, t = trajectory[-1]
    h_new, v_new, t_new = euler_step(h, v, t, g, dt)
    trajectory.append((h_new, v_new, t_new))

trajectory = np.array(trajectory)
```

Usually the cleanest strategy: use list.append() while the amount of data is growing, then convert once with np.array() for numerical analysis.

10. Recommended when N is KNOWN

```
N = 1000
trajectory = np.empty((N, 3))
trajectory[0, :] = [h0, v0, t0]

for i in range(1, N):
    h, v, t = trajectory[i-1, :]
    h_new, v_new, t_new = euler_step(h, v, t, g, dt)
    trajectory[i, :] = [h_new, v_new, t_new]
```

This is **preallocation**: reserve memory once and fill existing rows. It is usually much more efficient for large simulations.

11. Quick comparison

Method	Works in loop?	Efficiency	Recommendation
list.append() + np.array()	Yes	High	Best when N is unknown
np.vstack()	Yes	Medium/low	Very clear for learning
np.append(axis=0)	Yes	Medium/low	Valid; watch shapes
np.concatenate()	Yes	Medium/low	General and flexible
np.empty()/zeros()	Yes	High	Best when N is known

12. Mini cheat sheet

```
np.append(A, row, axis=0)      append along rows
np.vstack((A, row))            stack row below
np.concatenate((A,B), axis=0)  general concatenation
row.reshape(1, -1)             convert 1D row to 2D
row[np.newaxis, :]             same idea
np.empty((N, 3))               preallocate memory
A[i, :] = row                  fill existing row
```